المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⵓⵙⵙⵏⵉⵡⵉⵏ ⵜⵓⵏⵉⵡⵉⵏ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

COMPTE RENDU DU TP-3

1ÈRE ANNÉE – FILIÈRE INTELLIGENCE ARTIFICIELLE ET CYBERSÉCURITÉ

MODULE : ARCHITECTURE LOGICIELLE

Réalisé par :

ASIM ZINEB

Encadré par :

Pr. BE ELBAGHAZAOUI

Année unniversitaire:2025_2026

Introduction

Ce TP porte sur le déploiement d'un système distribué complet et l'observation de ses composants en interaction à travers des tests concrets.

Objectifs

Ce TP a pour objectif de mettre en place une architecture distribuée composée de six composants interconnectés : le client, le load balancer, le serveur, le cache, la base de données et le bus de messages. Il vise à comprendre le rôle de chacun de ces éléments à travers l'implémentation d'un flux complet de traitement d'une commande, depuis son envoi via une requête POST jusqu'à son stockage dans MongoDB, sa mise en cache avec Redis, puis l'envoi d'une notification via RabbitMQ. Enfin, le comportement de l'application distribuée est testé et observé à l'aide de Postman.

Outils et technologies utilisés Node.js / Express

- MongoDB
- Redis
- RabbitMQ
- Nginx
- Docker
- Postman

Architecture du système

L'architecture mise en place repose sur six composants principaux :

- **Client (Postman)** : envoie des requêtes HTTP (POST et GET).
- **Load Balancer (Nginx)** : répartit les requêtes entre plusieurs serveurs.
- **Serveur applicatif (Node.js / Express)** : traite la logique métier.
- **Base de données (MongoDB)** : assure le stockage persistant des commandes.
- **Cache (Redis)** : améliore les performances en évitant des accès répétés à la base.
- **Bus de messages (RabbitMQ)** : permet la communication asynchrone et l'envoi de notifications.

Le client envoie une requête qui passe d'abord par Nginx (load balancer), puis est traitée par une instance Node.js, avant d'être stockée dans MongoDB, mise en cache dans Redis et enfin envoyée à RabbitMQ pour la notification.

Structure du projet et initialisation

Le dossier principal tp3-architecture-distribuee a été créé avec la structure suivante :

```
tp3-architecture-distribuee/
├── docker-compose.yml    orchestre tous les conteneurs
├── nginx/
│   └── nginx.conf        configuration du load balancer
└── app/
    ├── package.json
    ├── server.js         serveur Node.js principal
    ├── consumer.js       consommateur de messages
    ├── routes/
    │   └── commandes.js routes API
    └── services/
        ├── mongodb.js    connexion MongoDB
        ├── redis.js      connexion Redis
        └── rabbitmq.js   connexion RabbitMQ
```

Le projet Node.js a été initialisé dans le dossier app pour générer le fichier package.json, pour installer les dépendances nécessaires.

Configuration des composants

1. Docker Compose

Le fichier docker-compose.yml permet de lancer MongoDB, Redis, RabbitMQ, Deux instances Node.js (app1 et app2), et Nginx.Chaque service est isolé dans un conteneur, mais tous communiquent entre eux via le réseau Docker.

```yaml
version: '3.8'
Run All Services
services:
  Run Service
  mongodb:
    image: mongo:6
    container_name: mongodb
    ports:
      - '27017:27017'
    volumes:
      - mongo_data:/data/db

  Run Service
  redis:
    image: redis:7-alpine
    container_name: redis
    ports:
      - '6379:6379'

  Run Service
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    ports:
      - '5672:5672'
      - '15672:15672'
    environment:
      RABBITMQ_DEFAULT_USER: admin
      RABBITMQ_DEFAULT_PASS: admin

  Run Service
  app1:
    build: ./app
    container_name: app1
    environment:
      PORT: 3001
      INSTANCE: 'Instance-1'
    ports:
      - '3001:3001'
    depends_on: [mongodb, redis, rabbitmq]

  Run Service
  app2:
    build: ./app
    container_name: app2
    environment:
      PORT: 3002
      INSTANCE: 'Instance-2'
    ports:
      - '3002:3002'
    depends_on: [mongodb, redis, rabbitmq]

  Run Service
  nginx:
    image: nginx:alpine
    container_name: nginx
    ports:
      - '80:80'
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
    depends_on: [app1, app2]

volumes:
  mongo_data:
```

2. Load Balancer (Nginx)

Nginx est configuré pour écouter sur le port 80 et distribuer les requêtes entre les deux instances Node.js. L'algorithme utilisé est Round-robin, ce qui signifie que les requêtes sont réparties de manière alternée entre les serveurs (app1 puis app2).

```
nginx > nginx.conf
events {
    worker_connections 1024;
}

http {
    upstream backend {
        server app1:3001;
        server app2:3002;
    }

    server {
        listen 80;
        location / {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

3. Serveur Node.js

Le serveur Express expose deux routes principales :

- POST /commandes : créer une commande
- GET /commandes : récupérer la liste des commandes

Chaque instance retourne également son nom pour vérifier le load balancing.

```
app > JS server.js > ...
const express = require('express');
const { connectMongo } = require('./services/mongodb');
const { connectRabbitMQ } = require('./services/rabbitmq');
const commandesRoutes = require('./routes/commandes');

const app = express();
const PORT = process.env.PORT || 3001;
const INSTANCE = process.env.INSTANCE || 'Instance-?';

app.use(express.json());

// Routes
app.use('/commandes', commandesRoutes);

// Route santé
app.get('/health', (req, res) => {
    res.json({ status: 'OK', instance: INSTANCE });
});

// Démarrage
async function start() {
    await connectMongo();
    await connectRabbitMQ();
    app.listen(PORT, () => {
        console.log(`🚀 ${INSTANCE} lancée sur le port ${PORT}`);
    });
}

start();
```

app > routes > JS commandes.js > ...

```javascript
const express = require('express');
const router = express.Router();
const { Commande } = require('../services/mongodb');
const redis = require('../services/redis');
const { publierCommande } = require('../services/rabbitmq');

const INSTANCE = process.env.INSTANCE || 'Instance-?';
const TTL = 30;
// POST /commandes — Créer une commande
router.post('/', async (req, res) => {
    try {
        const { produit, quantite, client } = req.body;

        // 1. Validation
        if (!produit || !quantite || !client) {
            return res.status(400).json({ erreur: 'Champs manquants' });
        }

        // 2. Sauvegarde MongoDB
        const commande = new Commande({ produit, quantite, client });
        await commande.save();

        // 3. Invalider le cache Redis
        await redis.del('toutes_commandes');

        // 4. Publier dans RabbitMQ
        publierCommande({ id: commande._id, produit, client });

        res.status(201).json({
            message: 'Commande créée',
            commande,
            traitee_par: INSTANCE
        });

    } catch (err) {
        res.status(500).json({ erreur: err.message });
    }
});
// GET /commandes — Lister toutes les commandes
router.get('/', async (req, res) => {
    try {
        // 1. Vérifier le cache Redis
        const cached = await redis.get('toutes_commandes');
        if (cached) {
            return res.json({
            return res.json({
                source: '⚡ CACHE Redis',
                traitee_par: INSTANCE,
                commandes: JSON.parse(cached)
            });
        }

        // 2. Si pas en cache → MongoDB
        const commandes = await Commande.find().sort({ date: -1 });

        // 3. Stocker en cache 30 secondes
        await redis.setex('toutes_commandes', TTL, JSON.stringify(commandes));

        res.json({
            source: '🗄 MongoDB',
            traitee_par: INSTANCE,
            commandes
        });

    } catch (err) {
        res.status(500).json({ erreur: err.message });
    }
});

module.exports = router;
```

4. Base de données MongoDB

MongoDB stocke les commandes de manière persistante. Le schéma Mongoose définit cinq champs : produit et client sont des chaînes obligatoires, quantite est un nombre obligatoire, statut a la valeur par défaut en_attente et date est initialisée automatiquement à la date courante.

```
app > services > JS mongodb.js > CommandeSchema > date > type
1   const mongoose = require('mongoose');
2
3   const CommandeSchema = new mongoose.Schema({
4       produit: { type: String, required: true },
5       quantite: { type: Number, required: true },
6       client: { type: String, required: true },
7       statut: { type: String, default: 'en_attente' },
8       date: { type: Date, default: Date.now }
9   });
10
11  const Commande = mongoose.model('Commande', CommandeSchema);
12
13  async function connectMongo() {
14      await mongoose.connect('mongodb://mongodb:27017/boutique');
15      console.log('✅ MongoDB connecté');
16  }
17
18  module.exports = { connectMongo, Commande };
```

5. Cache Redis

Redis est utilisé pour stocker temporairement les résultats des requêtes GET /commandes. Un TTL de 30 secondes est défini via la méthode setex pour expirer automatiquement les données. À chaque nouveau POST, le cache est invalidé via redis.del pour forcer une mise à jour lors du prochain GET.

```
app > services > JS redis.js > ...
1   const Redis = require('ioredis');
2
3   const redis = new Redis({ host: 'redis', port: 6379 });
4
5   redis.on('connect', () => console.log('✅ Redis connecté'));
6
7   module.exports = redis;
```

6. RabbitMQ

RabbitMQ permet d'envoyer un message dans la file commandes à chaque création de commande. Le fichier consumer.js simule un microservice de notification indépendant qui lit ces messages et affiche les détails de la commande en console, simulant ainsi l'envoi d'un email de confirmation.

app > services > rabbitmq.js > ...

```javascript
const amqp = require('amqplib');

let channel;
const QUEUE = 'commandes';

async function connectRabbitMQ() {
    const conn = await amqp.connect('amqp://admin:admin@rabbitmq');
    channel = await conn.createChannel();
    await channel.assertQueue(QUEUE, { durable: true });
    console.log('✅ RabbitMQ connecté');
}

function publierCommande(commande) {
    const msg = JSON.stringify(commande);
    channel.sendToQueue(QUEUE, Buffer.from(msg), { persistent: true });
    console.log('📤 Message publié dans RabbitMQ');
}

module.exports = { connectRabbitMQ, publierCommande };
```

app > consumer.js > ...

```javascript
const amqp = require('amqplib');

async function demarrerConsommateur() {
    const conn = await amqp.connect('amqp://admin:admin@rabbitmq');
    const channel = await conn.createChannel();
    const QUEUE = 'commandes';

    await channel.assertQueue(QUEUE, { durable: true });
    console.log('👂 Consommateur en attente de messages...');

    channel.consume(QUEUE, (msg) => {
        if (msg) {
            const commande = JSON.parse(msg.content.toString());
            console.log('');
            console.log('🔔 NOUVELLE COMMANDE REÇUE !');
            console.log('   Client  :', commande.client);
            console.log('   Produit :', commande.produit);
            console.log('   ID      :', commande.id);
            console.log('📧 Email de confirmation envoyé (simulé)');
            console.log('');
            channel.ack(msg);
        }
    });
}

// Attendre que RabbitMQ soit prêt
setTimeout(demarrerConsommateur, 5000);
```

7. Dockerfile

Crée l'image Docker de l'application Node.js en utilisant node:18-alpine comme base. Il définit /app comme répertoire de travail, copie d'abord package.json pour installer les dépendances, puis le reste du code, et démarre l'application avec node server.js. Cette méthode optimise le cache Docker.

```
app > Dockerfile > ...
1  FROM node:18-alpine
2  WORKDIR /app
3  COPY package*.json ./
4  RUN npm install
5  COPY . .
6  CMD ["node", "server.js"]
```

Lancement

Après exécution de la commande **docker-compose up --build**, les 7 conteneurs ont démarré avec succès.

```
PS C:\Users\zineb\tp-architectures\tp3-architecture-distribuee> docker-compose logs app1
>> docker-compose logs app2
app1  | ✅ Redis connecté
app1  | ✅ MongoDB connecté
app1  | ✅ RabbitMQ connecté
app1  | 🚀 Instance-1 lancée sur le port 3001
app2  | ✅ Redis connecté
app2  | ✅ MongoDB connecté
app2  | ✅ RabbitMQ connecté
app2  | 🚀 Instance-2 lancée sur le port 3002
```

Vérification

- ✓ **Nginx + Node.js**

http://localhost/health

JSON Données brutes En-têtes

Enregistrer Copier Tout réduire Tout développer Filtrer le JSON

```
status:    "OK"
instance:  "Instance-1"
```

- ✓ **Instance 1**

localhost:3001/health

Impression automatique

```
{"status":"OK","instance":"Instance-1"}
```

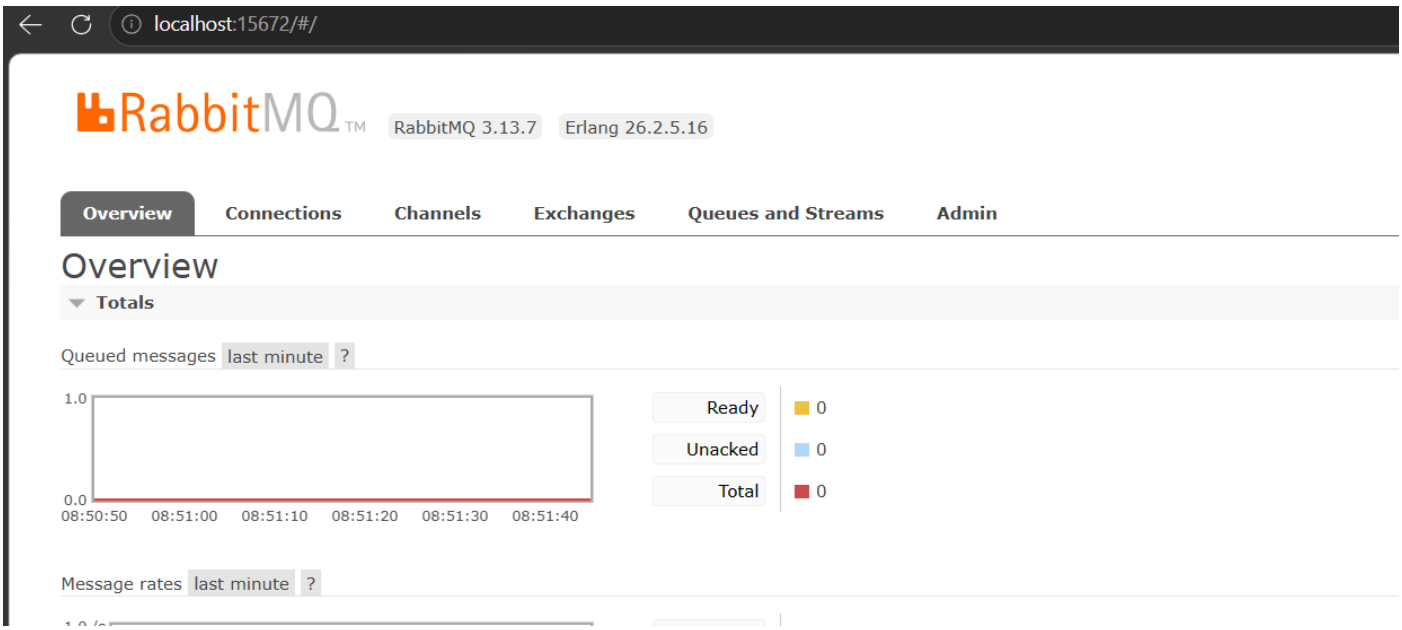
- ✓ **Instance 2**

localhost:3002/health

Impression automatique

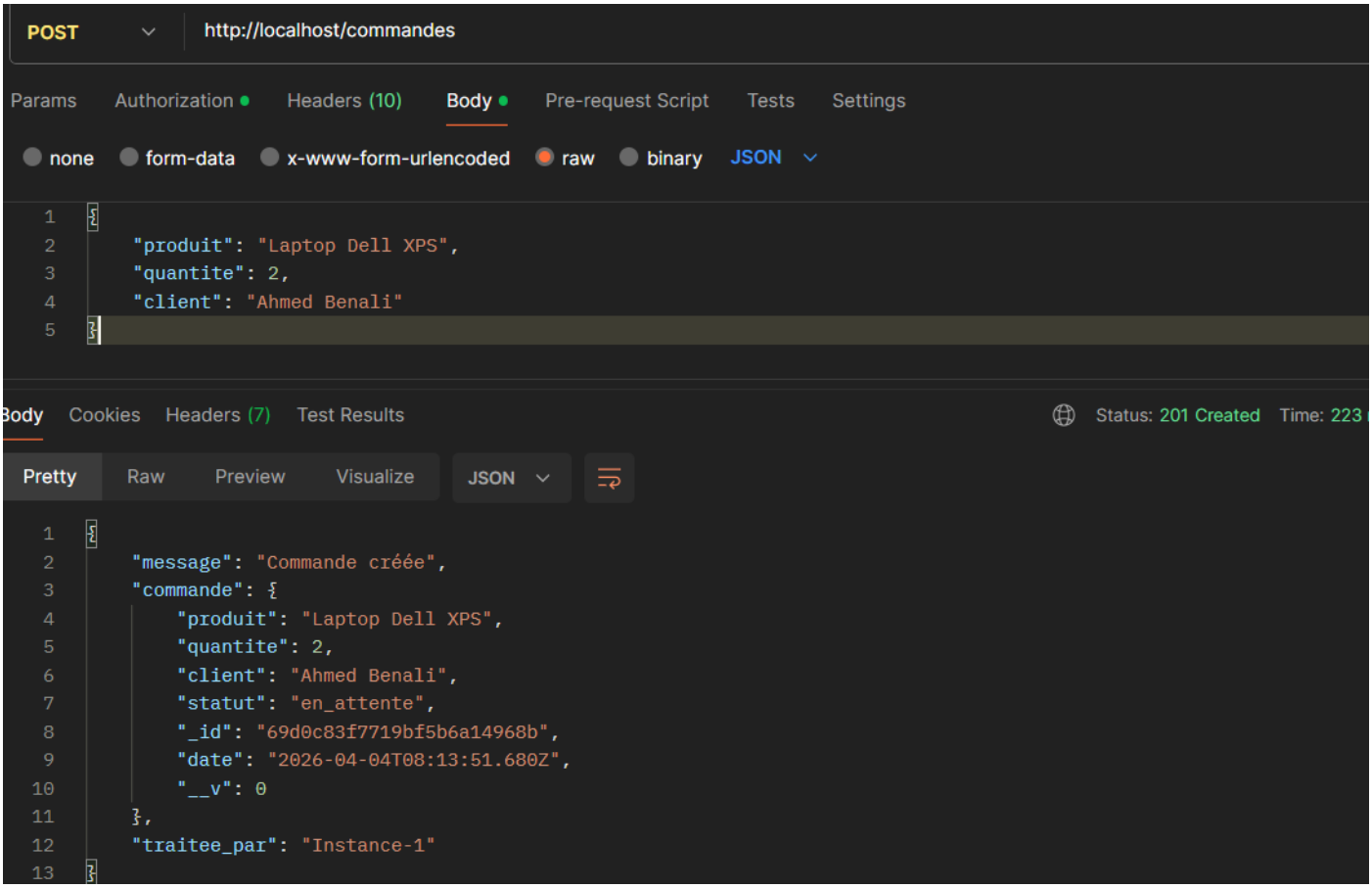
```
{"status":"OK","instance":"Instance-2"}
```

✓ **RabbitMQ**



Tests et résultats

✓ **Test 1 : Création d'une commande (POST)**



✓ **Test 2 : Récupération des commandes (GET)**

GET http://localhost/commandes

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary

This request does not have a body

Body Cookies Headers (7) Test Results Status: 200 OK Time: 175 ms Size: 465 B

Pretty Raw Preview Visualize JSON


```
{
    "source": "🗄 MongoDB",
    "traitee_par": "Instance-2",
    "commandes": [
        {
            "_id": "69d0c83f7719bf5b6a14968b",
            "produit": "Laptop Dell XPS",
            "quantite": 2,
            "client": "Ahmed Benali",
            "statut": "en_attente",
            "date": "2026-04-04T08:13:51.680Z",
            "__v": 0
        }
    ]
}
```


2eme GET

GET http://localhost/commandes

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary

This request does not have a body

Body Cookies Headers (7) Test Results Statu

Pretty Raw Preview Visualize JSON


```
{
    "source": "⚡ CACHE Redis",
    "traitee_par": "Instance-1",
    "commandes": [
        {
            "_id": "69d0c83f7719bf5b6a14968b",
            "produit": "Laptop Dell XPS",
            "quantite": 2,
            "client": "Ahmed Benali",
            "statut": "en_attente",
            "date": "2026-04-04T08:13:51.680Z",
            "__v": 0
        }
    ]
}
```


Le deuxième appel GET retourne source: " CACHE Redis", confirmant que Redis met bien en cache les résultats et les sert directement sans interroger MongoDB.

✓ **Test 3 : Load balancing**

POST http://localhost/commandes

Params Authorization Headers (10) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
{
    "produit": "iPhone 15",
    "quantite": 1,
    "client": "zineb"
}
```

Body Cookies Headers (7) Test Results Status: 201 Creat

Pretty Raw Preview Visualize JSON

```
{
    "message": "Commande créée",
    "commande": {
        "produit": "iPhone 15",
        "quantite": 1,
        "client": "zineb",
        "statut": "en_attente",
        "_id": "69d0cad66d949b317c700e82",
        "date": "2026-04-04T08:24:54.688Z",
        "__v": 0
    },
    "traitee_par": "Instance-2"
}
```

2eme POST

POST http://localhost/commandes

Params Authorization Headers (10) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
{
    "produit": "iPhone 15",
    "quantite": 1,
    "client": "zineb"
}
```

Body Cookies Headers (7) Test Results Status: 201 Created Time:

Pretty Raw Preview Visualize JSON

```
{
    "message": "Commande créée",
    "commande": {
        "produit": "iPhone 15",
        "quantite": 1,
        "client": "zineb",
        "statut": "en_attente",
        "_id": "69d0cb766d949b317c700e8a",
        "date": "2026-04-04T08:27:34.443Z",
        "__v": 0
    },
    "traitee_par": "Instance-2"
}
```

Quatre requêtes POST successives ont été envoyées via Nginx, le champ traitee_par alterne entre Instance-1 et Instance-2 à chaque requête, confirmant que Nginx répartit la charge en Round-robin entre les deux instances Node.js.

Réponses aux questions

Question 1 :

Le composant qui reçoit en premier les requêtes est **Nginx (Load Balancer)**.
Il utilise l'algorithme **Round-robin** (par défaut dans Nginx), qui distribue les requêtes en alternance entre les serveurs :

- ✓ Requête 1 → app1:3001
- ✓ Requête 2 → app2:3002
- ✓ Requête 3 → app1:3001 … etc.

(Captures du Test 3)

Question 2 :

Si app1 tombe en panne, Nginx continue de fonctionner et redirige les requêtes vers app2.
Cela est possible car plusieurs instances sont disponibles, ce qui assure la continuité du service.
C'est le principe de la haute disponibilité : Nginx détecte que app1 ne répond plus et l'exclut automatiquement du pool upstream. Le système reste opérationnel grâce à app2.

Question 3 :

Le premier GET interroge MongoDB (source persistante), ce qui est plus lent.
Le deuxième GET utilise Redis (cache), ce qui est beaucoup plus rapide.

Question 4 :

Le cache Redis est invalidé après un POST pour éviter d'avoir des données obsolètes.
Cela garantit que les prochaines requêtes GET retournent des données à jour.

Question 5 :

RabbitMQ introduit un découplage asynchrone entre le serveur Node.js et le service de notification. Sans RabbitMQ, si le service d'envoi d'email tombe en panne, la commande échouerait. Avec RabbitMQ :

- Le serveur publie le message dans la file et répond immédiatement au client (201)
- Si le consommateur est temporairement indisponible, le message reste persistant dans la file (option durable : true) et sera traité dès que le consommateur redémarre
- Le système est donc résilient aux pannes partielles

Question 6 :

Il faut modifier le bloc upstream backend dans nginx/nginx.conf en ajoutant une ligne ; **server app3:3003;** et ajouter le service app3 dans le docker-compose.yml de la même façon que app1 et app2, avec PORT: 3003.

Conclusion

Ce TP a permis de comprendre concrètement le fonctionnement d'une architecture distribuée moderne.
L'utilisation combinée de Nginx, MongoDB, Redis et RabbitMQ permet d'obtenir un système performant, scalable et résilient.
Cette évolution met en évidence les avantages d'une architecture distribuée en termes de scalabilité, de résilience et de performance par rapport à une architecture monolithique.